1.output

Algorithm: Insertion Sort (basic operation: a[j] > key)

Trials per size: 30 CPU time supported: true

n	mean ops	best ops	worst ops	mean/n^2	CPU MS	wall MS
250	15755	249	31125	0.2521	0.000	0.007
500	62921	499	124750	0.2517	0.000	0.026
1000	251408	999	499500	0.2514	0.000	0.117
2000	1001217	1999	1999000	0.2503	1.042	0.407
4000	3989988	3999	7998000	0.2494	2.604	0.803
8000	16001678	7999	31996000	0.2500	5.729	4.510

2.theorectical Complexity

Best case (list already sorted): so, each (i) need only one comparison

$C_{best}(n) = \Sigma\ (i = 1 \text{ to } n-1)\ 1 = n - 1 \in$ big-Theta (n)

Worst case (list is flipped/ reverse order): so, each (i) need (i) comparison

$C_{worst}(n) = \Sigma\ (i = 1 \text{ to } n-1)\ i = n\ (n - 1)/2 \in$ big-Theta (n^2)

Average case (random list): so, about half of the number before (i) are bigger, so each (i) must be i/2

$C_{avg}(n) \approx \Sigma\ (i = 1 \text{ to } n-1)\ i/2 = n\ (n - 1)/4 \approx n^2/4 \in$ big-Theta (n^2)

3. empirical complexity

To find the empirical complexity, we do (mean/n^2)

15755/250^2 = 15755/62500 ≈ 0.2521

62921/500^2= 62921/250000 ≈ 0.2517

251408/1000^2 = 251408/1000000 ≈ 0.2514

1001217/2000^2 = 1001217/4000000 ≈ 0.2503

3989988/4000^2 = 3989988/16000000 ≈ 0.2494

16001678/8000^2 = 16001678/64000000 ≈ 0.2500

The value stays close to 0.25 for all input sizes we have. This shows us that the number of operations grows close to n^2

C_empirical (n)∈ big-Theta (n^2)

4. CPU time vs wall-clock time

So, we were able to collect both CPU time and wall-clock time. So, the CPU time is the amount of time my computer/other computer spends working on the sorting algorithm. Wall-clock time is the total time that passes while the sorting algorithm is running. This

matters because wall-clock time can be affected by other programs running on my computer/other computers.